COMS 5790: Project 3

Giuliano Pitari
Iowa State University, Ames, IA
Politecnico di Torino, Turin, Italy
gpit8@iastate.edu
Kaggle Username: Giuliano Pitari

1 Method

1.1 Problem Formulation

This project addresses short-answer question answering over four lecture transcripts in ecology, statistics, physics, and computer science. The test set contains 110 questions, including fill-in-the-blank and yes/no questions. Because the transcripts include counterfactual statements, answers must be grounded in the supplied documents rather than external knowledge. The system therefore retrieves lecture passages for each question and asks a language model to produce a concise answer. The required submission is a CSV with `ID` and `answer` columns. Kaggle evaluates submissions using ROUGE-L F1 [4; 11]; the stated passing threshold is 0.35.

1.2 Transcript Preprocessing

The transcript formatting contains line breaks that do not reliably indicate sentence or paragraph boundaries. I replace runs of whitespace with a single space, preserving the transcript text while avoiding treating pauses as document structure; the cleaned text has 76,034 characters. I then split it into 224 chunks of 400 characters with a 50-character overlap using LangChain's [9] recursive character splitter. Since no newlines remain after cleaning, the splitter breaks the text at periods and, when needed, at spaces. The overlap is intended to preserve relevant context when a supporting fact falls near a chunk boundary. The four lectures are concatenated without a separator, so a chunk near a lecture boundary may contain text from two lectures.

1.3 Dense Retrieval

The selected system, submission C, uses the same dense retrieval pipeline as the initial submission A. Each chunk is encoded with the `all-MiniLM-L6-v2` sentence-embedding model [5]. The embeddings are indexed in FAISS, a library for efficient similarity search [2]. For each question, the retriever returns the three chunks with the highest semantic similarity. These chunks are concatenated and provided to the generator as the question-specific context. This retrieve-then-generate architecture follows the general retrieval-augmented generation approach [3].

1.4 Baseline A and Selected System C

Submission A is the baseline. It uses `google/flan-t5-large` with dense retrieval and the original prompt, which contains five paraphrased answer-format examples and applies a general yes/no instruction. A achieved a public-leaderboard score of 0.51248.

Submission C retains A's transcript preprocessing, chunking, embedding model, FAISS index, and retrieval of three dense chunks. It changes the generation setup: C uses `google/flan-t5-xl` [1; 8], the five instructor-provided question-answer examples while omitting the example for the current question, and a yes/no instruction only for questions classified as binary. C also applies answer post-processing and achieved a public-leaderboard score of 0.65260.

Because the model, prompt, and post-processing differ between A and C, this comparison does not isolate the effect of any single change.

For each question, the prompt provides the retrieved lecture context and instructs the model to give a short answer based only on that context. It includes the instructor-provided examples, except for the example matching the current question. The prompt requests "Yes" or "No" only for questions classified as binary (a question starting with an auxiliary verb or containing "true?"; this rule identifies 3 of the 110 test questions), asks for only the missing words in fill-in-the-blank questions, and specifies "I don't know" when the context is insufficient. The model generates up to 30 new tokens

System	Generator	Retrieval	Score
A	Flan-T5-large	Dense	0.51248
B	Flan-T5-large	Dense	0.47814
C	Flan-T5-XL	Dense	**0.65260**
D	Flan-T5-XL	Dense + BM25	0.64250

Table 1: Public leaderboard scores (ROUGE-L F1). A uses the original prompt and no answer cleaning; B, C, and D use the revised prompt, answer cleaning, and a 600-token prompt limit.

with greedy decoding in float32 using Hugging Face Transformers [7]. Post-processing removes an optional answer label and surrounding punctuation, maps empty outputs to "I don't know," and reduces a yes/no response to "Yes" or "No" only when the question is classified as binary.

In B, C, and D the prompt is limited to a budget of 600 tokens: if it is longer, retrieved chunks are removed from the end of the context one at a time until it fits; the question and instructions are never truncated. System A applies no such limit. The instructions and examples alone take about 290 tokens, so a prompt with three 400-character chunks is close to the budget and chunks may be removed.

1.5 Results

B and D are variants of A and C described in Appendix A: B uses A's model with C's prompt and cleaning, and D adds two BM25 chunks to C. Table 1 reports the leaderboard score of the four systems. Submission C achieved the highest score, 0.65260, above the course's stated passing threshold of 0.35. ROUGE-L measures the longest common subsequence between a candidate and reference text [4].

B and C differ only in the generator. Moving from Flan-T5-large to Flan-T5-XL raises the score by 0.17446, the largest effect observed, and the number of "I don't know" answers drops from 19 to 13. On the five known questions, the only difference in ROUGE-L comes from question 2, where the larger model writes "arithmetic logic unit" instead of the abbreviation (Table 3). C and D differ only in the retriever, and adding two BM25 chunks lowers the score by 0.01010; as discussed in Appendix A, this comparison is weak. A and B change the prompt, the answer cleaning, and the token limit together, so the drop from 0.51248 to 0.47814 cannot be attributed to a single component. Each score comes from a single run, so differences of about one point are not conclusive. The five instructor-provided answers are too few to estimate performance on the 110 questions, and A's prompt contained those five answers as examples, so its score on them is not a clean estimate. Appendix A gives further diagnostics and an analysis of failure cases.

References

[1] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, and others. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.

[2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

[4] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.

[5] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992.

[6] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

[7] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, and others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, pages 38–45.

[8] Google. 2022. Flan-T5-XL model card. `https://huggingface.co/google/flan-t5-xl`.

[9] LangChain. Text splitter documentation. `https://python.langchain.com/docs/concepts/text_splitters/`.

[10] Dorian Brown. `rank_bm25` package. `https://github.com/dorianbrown/rank_bm25`.

[11] Google Research. `rouge-score` package. `https://pypi.org/project/rouge-score/`.

A Discussion: Advanced Experiments and Critical Analysis

I compared the four system configurations of Table 1. A is the initial Flan-T5-large system with the original prompt and dense retrieval. B retains the model and dense retriever but uses a revised prompt and answer cleaning. C uses Flan-T5-XL with the revised prompt and dense retrieval. D uses the C configuration and adds two chunks retrieved with BM25 [6], through LangChain's retriever, which wraps the `rank_bm25` package [10], to the three dense-retrieved chunks. Table 2 summarizes diagnostics computed on the generated answers.

	A	B	C	D
ROUGE-L (5 known)	0.40	0.20	0.40	0.40
Yes/No to non-binary	4	0	0	0
"I don't know"	19	19	13	13
Mean words/answer	3.2	3.2	3.6	3.7

Table 2: Diagnostics of the four submissions. ROUGE-L is the mean F1 on the five instructor-provided answers (not clean for A, whose prompt contained them); "Yes/No to non-binary" counts non-binary questions answered with "Yes" or "No."

B illustrates a trade-off: its public score was lower than A's, decreasing from 0.51248 to 0.47814, while the number of non-binary questions answered with "Yes" or "No" decreased from four to zero. Both systems produced 19 "I don't know" answers. Mean ROUGE-L F1 on the five known answers decreased from 0.40 for A to 0.20 for B. Thus, B improved this particular answer-format behavior but did not improve the overall score in this submission. Because B changed both the prompt and answer cleaning, their individual effects cannot be separated. The ROUGE-L drop on the five known answers is partly an artifact of the comparison: A's prompt contained those five answers, whereas B's prompt excludes the example of the current question.

D tests whether adding sparse retrieval improves on C's dense retrieval. Its public score was 0.64250, a decrease of 0.01010 compared with C. Both C and D had a mean ROUGE-L F1 of 0.40 on the five known answers, 13 "I don't know" answers, and no "Yes" or "No" answers to non-binary questions. D appends the two BM25 chunks after the three dense chunks, and the prompt-length limit removes chunks from the end of the context, so BM25 chunks are the first to be discarded when the prompt exceeds the budget. Since the instructions and examples already take about 290 tokens, this probably happens often, which makes D a weak test of hybrid retrieval; I did not measure how many BM25 chunks reached the model. The difference comes from a single run and the results do not establish its cause. A fairer test would place the BM25 chunks first, shorten the examples, or rerank the union of the candidates.

Among the tested configurations, C achieved the highest observed public score. This is an empiri-

ID	Expected	A	B	C, D
1	Representative	subset	subset	I don't know
2	Arithmetic logic unit	ALU	ALU	arithmetic logic unit
3	int	int	I-N-T	I-N-T
4	1993	1993	1993	1993
5	Green laser	No	I don't know	I don't know

Table 3: Predictions for the five instructor-provided answers. C and D produce identical answers on these questions.

cal result for these submissions, not evidence that dense retrieval is generally superior to hybrid retrieval. The comparisons also do not isolate every change: A-to-B changes both prompt and cleaning, while B-to-C changes the generator model. Therefore, the score differences cannot be attributed with certainty to any single change.

One execution emitted a warning for a 515-token input, exceeding the model's stated 512-token limit. The configured prompt budget was 600 tokens, so it did not consistently enforce that limit; in system A, which has no budget, 71 of the 110 prompts exceeded 512 tokens (maximum 578). T5 uses relative position encodings and can process longer inputs, but it was trained on 512 tokens, so the extra length may degrade performance. A more robust implementation should enforce the tokenizer's maximum input length while preserving the question and instructions; shorter instructions or examples would free room for more context. The reported Kaggle scores belong to the submitted runs and should not be assumed to remain unchanged after modifying the code.

Failure analysis. C answers two of the five known questions correctly (Table 3). On question 3, A copied "int" from an example in its prompt, whereas B, C, and D spelled the answer out as "I-N-T," possibly because of the rule against abbreviations. On question 1, C answers "I don't know"; in a retrieval test on this question, the retrieved chunks discuss unbiased samples and the relation between sample and population but do not contain the expected word "representative," which suggests a retrieval failure. Five questions are too few to generalize, and I did not check the remaining "I don't know" answers against the transcripts.

B AI Experience

I used an AI assistant to help inspect notebook outputs, debug implementation issues, and organize the comparison between system variants. I ran the experiments and checked the resulting predictions and Kaggle scores. The assistant was useful for generating hypotheses about failure cases, but its suggestions had to be checked against actual outputs; for example, prompt changes that seemed reasonable did not consistently improve the measured score.